

프로젝트 중간 보고

2025107032 김나연

1. 작품명

LangGraph 의 Human-in-the-loop 구조를 활용한 주니어 개발자 맞춤형 지능형 커리어 로드맵 생성 에이전트

2-1. 백엔드 아키텍처 (FastAPI & LangGraph)

- **FastAPI 기반 REST API:** 고성능 비동기 프레임워크인 FastAPI 를 사용하여 에이전트의 추론 과정을 실시간으로 서빙합니다.
- **Swagger UI 활용:** API 설계 단계에서부터 Swagger 를 활용하여, 에이전트의 상태(State) 값 전달 및 Human-in-the-loop 피드백 엔드포인트를 시각적으로 문서화하고 테스트 중입니다.
- **Stateful Thread 관리:** uuid 를 이용한 thread_id 기반의 세션 관리를 통해, 각 사용자별로 독립적인 로드맵 생성 상태를 유지합니다
- **Human-in-the-loop (HITL) 엔진:** LangGraph 의 interrupt 기능을 활용하여, AI 가 계획을 세운 후 사용자의 승인을 기다리는 'Check-and-Balance' 시스템을 구현하였습니다.
- **SQLite 기반 체크포인트 영속화:** LangGraph 의 AsyncSqliteSaver 를 체크포인트로 적용하여, 에이전트의 전체 상태(기술 스택 분석 결과, 로드맵 초안, 수정 이력)를 SQLite 파일(checkpoints.db)에 영속화하였습니다. 이를 통해 서버가 재시작되더라도 기존 세션이 유지되며, 사용자는 중단된 지점부터 피드백 과정을 이어갈 수 있습니다. 기존 인메모리 방식(MemorySaver)의 휘발성 한계를 해결하였습니다.
- **비동기 아키텍처 일관성:** FastAPI 의 비동기 이벤트 루프에 맞춰 LangGraph 의 ainvoke / aget_state 패턴을 적용하였습니다. 이를 통해 LLM 추론

대기 중에도 다른 사용자의 요청을 동시에 처리할 수 있어, 복수 사용자 환경에서의 응답 지연을 방지합니다.

2-2 핵심 API 로직 상세

- **/start 엔드포인트:** 사용자의 이력서를 입력받아 그래프를 최초 실행합니다.
 - `career_agent.invoke` 를 통해 분석 및 계획 수립 노드를 거친 후, `interrupt` 지점에서 멈춰진 상태의 초안(`roadmap_draft`)을 사용자에게 반환합니다.
- **/feedback 엔드포인트:** 사용자의 피드백을 `Command(resume=req.feedback)`를 통해 에이전트에게 전달합니다.
 - 에이전트는 사용자의 의도를 반영하여 로드맵을 수정하거나(`revision_requested`), 최종 승인(`approved`) 상태로 전이합니다.
 - 그래프의 스냅샷(`get_state`)을 분석하여 현재 프로세스가 종료되었는지, 추가 수정이 필요한 대기 상태인지를 자동으로 판단합니다.
- **/history/{thread_id} 엔드포인트:** 특정 세션의 로드맵 수정 이력을 조회합니다.
 - `AgentState` 에 `revision_history` 필드를 추가하고, `LangGraph` 의 `Annotated Reducer` 패턴을 적용하여 `reviser` 노드가 실행될 때마다 수정 전 로드맵이 자동으로 이력에 누적되도록 설계하였습니다.
 - 현재 로드맵(`current_roadmap`), 총 수정 횟수(`revision_count`), 버전별 이전 로드맵 목록(`revision_history`)을 JSON 으로 반환합니다.
 - 이 이력 데이터는 향후 구현 예정인 Time-travel 기능(로드맵 버전 비교 및 롤백)의 데이터 기반이 됩니다.

2-3. 프론트엔드 설계 (React & CSS)

- Interactive Roadmap UI: 에이전트가 생성한 로드맵(JSON 데이터)을 바탕으로 동적인 타임라인 형태의 UI 를 렌더링합니다.
- Human-in-the-loop 인터랙션: 사용자가 특정 학습 카드(Node)를 클릭하여 수정 사항을 입력하면, 이를 FastAPI 의 /feedback 엔드포인트로 전송하여 에이전트가 로드맵의 해당 부분만 다시 그리도록 구현합니다.
- State 시각화: 에이전트가 현재 어느 단계(분석 중, 계획 중, 피드백 대기 중)에 있는지 상태를 실시간으로 표시하여 사용자 경험을 향상시킵니다.

3. 현재 개발 단계

- [완료] LangGraph 기반의 상태 관리 및 피드백 순환 로직 개발.
- [완료] FastAPI 를 이용한 에이전트 컨트롤러 및 HITL 인터럽트 엔드포인트 구현.
- [진행 중] 사용자의 기술 스택 분석 데이터 보존을 위한 db 연동 작업.

4. 향후 계획

(~ 5/10): 에이전트의 추론 과정을 시각화하여 보여주는 React 기반 대시보드 UI 개발.

(~ 5/20): 로드맵 수정 이력을 확인할 수 있는 Time-travel 기능(LangGraph Checkpointer 활용) 고도화.

(~ 5/31): 실제 주니어 개발자들을 대상으로 한 사용성 평가 및 최종 결과물 제출.